

# JAVA PROGRAMMING CONCEPTS

IIT Madras B.Sc Programming & Data Science — Week 2 Master Notes

Instructor: Prof. Madhavan Mukund (Chennai Mathematical Institute)

Comprehensive Master Reference Manual

## Contents

|          |                                                                                              |          |
|----------|----------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>IITM Java Course — Week 2 Combined Master Notes</b>                                       | <b>2</b> |
| <b>2</b> | <b>IITM Java Course — Week 2, Lecture 7: A First Taste of Java</b>                           | <b>2</b> |
| 2.1      | Key Technical Terms Dictionary (Defined Upon First Appearance) . . . . .                     | 2        |
| 2.1.1    | Prof. Mukund's Words of Wisdom & Best Practices (Lecture 7) . . . . .                        | 2        |
| 2.2      | 1. Anatomy of a First Java Program . . . . .                                                 | 2        |
| 2.2.1    | Breakdown of Keyword Signatures: . . . . .                                                   | 2        |
| 2.3      | 2. The Compilation & Execution Pipeline . . . . .                                            | 3        |
| 2.3.1    | Command-Line Compilation & Execution Cycle: . . . . .                                        | 3        |
| 2.3.2    | Executable Code Example: Hello World & Command-Line Arguments . . . . .                      | 3        |
| 2.4      | 3. Comprehensive Summary & Key Takeaways . . . . .                                           | 3        |
| <b>3</b> | <b>IITM Java Course — Week 2, Lecture 8: Basic Datatypes in Java</b>                         | <b>3</b> |
| 3.1      | Key Technical Terms Dictionary (Defined Upon First Appearance) . . . . .                     | 3        |
| 3.1.1    | Prof. Mukund's Words of Wisdom & Best Practices (Lecture 8) . . . . .                        | 4        |
| 3.2      | 1. The Eight Primitive Datatypes in Java . . . . .                                           | 4        |
| 3.3      | 2. Two's Complement Formula & Range Bounds . . . . .                                         | 4        |
| 3.3.1    | Executable Code Example: Overflow Wrap-Around & Type Conversion . . . . .                    | 4        |
| 3.4      | 3. Comprehensive Summary & Key Takeaways . . . . .                                           | 5        |
| <b>4</b> | <b>IITM Java Course — Week 2, Lecture 9: Control Flow Constructs in Java</b>                 | <b>5</b> |
| 4.1      | Key Technical Terms Dictionary (Defined Upon First Appearance) . . . . .                     | 5        |
| 4.1.1    | Prof. Mukund's Words of Wisdom & Best Practices (Lecture 9) . . . . .                        | 5        |
| 4.2      | 1. Conditional Branching (if, else if, switch) . . . . .                                     | 6        |
| 4.2.1    | Executable Code Example: Conditionals, Switch, and Loop Constructs . . . . .                 | 6        |
| 4.3      | 2. Comprehensive Summary & Key Takeaways . . . . .                                           | 6        |
| <b>5</b> | <b>IITM Java Course — Week 2, Lecture 10: Defining Classes and Objects in Java</b>           | <b>7</b> |
| 5.1      | Key Technical Terms Dictionary (Defined Upon First Appearance) . . . . .                     | 7        |
| 5.1.1    | Prof. Mukund's Words of Wisdom & Best Practices (Lecture 10) . . . . .                       | 7        |
| 5.2      | 1. Class Structure: Instance Variables & Methods . . . . .                                   | 7        |
| 5.2.1    | Executable Code Example: Bank Account Class with Overloaded Constructors & Methods . . . . . | 7        |
| 5.3      | 2. Comprehensive Summary & Key Takeaways . . . . .                                           | 8        |
| <b>6</b> | <b>IITM Java Course — Week 2, Lecture 11: Basic Input and Output in Java</b>                 | <b>8</b> |
| 6.1      | Key Technical Terms Dictionary (Defined Upon First Appearance) . . . . .                     | 9        |
| 6.1.1    | Prof. Mukund's Words of Wisdom & Best Practices (Lecture 11) . . . . .                       | 9        |
| 6.2      | 1. Reading User Input with <code>Scanner</code> & Formatted Output . . . . .                 | 9        |
| 6.2.1    | Format Specifiers Matrix for <code>System.out.printf()</code> : . . . . .                    | 9        |
| 6.2.2    | Executable Code Example: Scanner Parsing & Formatted Output . . . . .                        | 9        |
| 6.3      | 2. Comprehensive Summary & Key Takeaways . . . . .                                           | 10       |

# 1 IITM Java Course — Week 2 Combined Master Notes

**Course:** Programming Concepts Using Java

**Instructor:** Prof. Madhavan Mukund (CMI / IIT Madras)

**Target:** Comprehensive textbook-grade study guide covering Lectures 07 to 11.

## 2 IITM Java Course — Week 2, Lecture 7: A First Taste of Java

**Instructor:** Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras)

**Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

---

### 2.1 Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Java Virtual Machine (JVM):** An abstract computing machine that executes compiled Java Bytecode (.class files) on any underlying operating system.
- **Java Bytecode:** Platform-independent intermediate instruction code generated by the `javac` compiler.
- **javac:** The Java Compiler command-line tool that compiles .java source files into .class bytecode.
- **java:** The Java Application Launcher command-line tool that executes the JVM on a compiled class.
- **WORA (“Write Once, Run Anywhere”):** Java’s core architectural principle enabling bytecode to run unmodified across Linux, Windows, and macOS.
- **main() Method:** The mandatory entry-point method signature (`public static void main(String[] args)`) invoked by the JVM to start program execution.
- **String[] args:** Command-line arguments passed as an array of strings to the `main()` method.

#### 2.1.1 Prof. Mukund’s Words of Wisdom & Best Practices (Lecture 7)

1. **The Class-to-Filename Strict Constraint:** \* “In Java, if a class is declared *public*, the file name *MUST* match the class name exactly, including case sensitivity (e.g. `public class HelloWorld` must sit in `HelloWorld.java`). Failing to match class name and file name causes an immediate compilation failure.”

2. **Why main MUST Be Static:** \* “When the Java Virtual Machine starts your program, no objects have been created in heap memory yet! Marking `main()` as *static* allows the JVM to invoke the entry method directly on the class template without needing to instantiate an object first.”

3. **The WORA Bytecode Bridge:** \* “C and C++ compile directly to native machine code for a specific CPU architecture (x86\_64, ARM). Java compiles to intermediate JVM Bytecode. As long as a system has a JVM installed, the same .class file runs seamlessly without re-compilation.”

### 2.2 1. Anatomy of a First Java Program

Every line of executable code in Java must reside inside a **Class** definition.

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

#### 2.2.1 Breakdown of Keyword Signatures:

1. **public:** Access modifier making the class visible to the JVM host environment.
2. **class:** Keyword defining a new object template.
3. **static:** Indicates the method belongs to the class itself, not an instantiated object instance.
4. **void:** Indicates the method does not return any value.
5. **main:** The designated name for the application entry point.
6. **String[] args:** Captures command-line arguments passed at execution time.

## 2.3 2. The Compilation & Execution Pipeline

```
graph LR;
  A["HelloWorld.java (Source Code)"] -->|javac Compiler| B["HelloWorld.class (Bytecode)"];
  B -->|JVM Interpreter / JIT| C["Native Machine Code (CPU Execution)"];
```

### 2.3.1 Command-Line Compilation & Execution Cycle:

```
# Step 1: Compile source code into bytecode
javac HelloWorld.java

# Step 2: Launch the JVM and execute the bytecode
java HelloWorld
```

### 2.3.2 Executable Code Example: Hello World & Command-Line Arguments

```
public class FirstProgram {
    public static void main(String[] args) {
        System.out.println("Welcome to Java Programming (IITM B.Sc Course)!");

        // Simulating Command-line Arguments
        String[] demoArgs = {"Java", "IITM", "2026"};
        System.out.println("Passed Arguments count: " + demoArgs.length);
        for (int i = 0; i < demoArgs.length; i++) {
            System.out.println("Arg[" + i + "]: " + demoArgs[i]);
        }
    }
}

FirstProgram.main(new String[]{});
```

## 2.4 3. Comprehensive Summary & Key Takeaways

1. **Class Container:** In Java, all code must reside inside a class.
2. **public static void main:** Mandatory entry point invoked directly by the JVM.
3. **Bytecode Portability:** javac produces .class bytecode, providing cross-platform independence.

## 3 IITM Java Course — Week 2, Lecture 8: Basic Datatypes in Java

**Instructor:** Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras)

**Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

### 3.1 Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Primitive Datatypes:** Built-in fundamental datatypes in Java stored directly on the Stack frame rather than dynamic Heap references.
- **Two's Complement:** A binary representation system for signed integers where the most significant bit represents  $-2^{n-1}$ , providing range  $[-2^{n-1}, 2^{n-1} - 1]$ .
- **IEEE 754 Floating-Point:** Standardized representation for real numbers (float and double) using sign, exponent, and mantissa bits.
- **UTF-16 Unicode:** A 16-bit character encoding system allowing Java's char type to represent international characters, symbols, and scripts.
- **Implicit Widening Conversion:** Automatic type conversion performed by the compiler when converting a smaller datatype to a larger datatype (e.g. int  $\rightarrow$  double).

- **Explicit Narrowing Conversion (Casting):** Manual type conversion (`(int) myDouble`) required when assigning a larger datatype to a smaller datatype, incurring precision loss or truncation.
- **Integer Overflow Wrap-Around:** A condition where exceeding the maximum value of a signed integer wraps around to the negative minimum bound.

### 3.1.1 Prof. Mukund's Words of Wisdom & Best Practices (Lecture 8)

**1. The Danger of Integer Overflow Wrap-Around:** \* “In Java, integer arithmetic does NOT throw a runtime error when it exceeds bounds! Adding 1 to `Integer.MAX_VALUE` (2147483647) wraps around silently to `Integer.MIN_VALUE` (-2147483648). Always ensure your variable range matches your computational domain to prevent catastrophic wrap-around bugs.”

**2. Why boolean is NOT an Integer in Java:** \* “In Python and C, booleans are integers (0 is False, 1 is True). In Java, `boolean` is a distinct type that CANNOT be implicitly or explicitly cast to an integer (if `x = 1`) will throw a compile error). This design decision eliminates a huge category of assignment-in-conditional bugs.”

**3. UTF-16 Unicode vs. ASCII:** \* “Java was designed for global applications. While C uses 8-bit ASCII (`char`), Java uses 16-bit UTF-16 Unicode for `char`, allowing native representation of Indian languages (Devanagari, Tamil) and international scripts directly in source text.”

## 3.2 1. The Eight Primitive Datatypes in Java

Java provides 8 fundamental primitive datatypes categorized into 4 groups:

| Category              | Datatype             | Size (Bits) | Minimum Value                      | Maximum Value                     | Default Value       |
|-----------------------|----------------------|-------------|------------------------------------|-----------------------------------|---------------------|
| <b>Integral</b>       | <code>byte</code>    | 8 bits      | -128                               | 127                               | 0                   |
|                       | <code>short</code>   | 16 bits     | -32,768                            | 32,767                            | 0                   |
|                       | <code>int</code>     | 32 bits     | -2,147,483,648                     | 2,147,483,647                     | 0                   |
|                       | <code>long</code>    | 64 bits     | -9,223,372,036,854,775,808         | 9,223,372,036,854,775,807         | 0                   |
| <b>Floating-Point</b> | <code>float</code>   | 32 bits     | $\approx \pm 1.4 \times 10^{-45}$  | $\approx \pm 3.4 \times 10^{38}$  | 0.0f                |
|                       | <code>double</code>  | 64 bits     | $\approx \pm 4.9 \times 10^{-324}$ | $\approx \pm 1.8 \times 10^{308}$ | 0.0d                |
| <b>Textual</b>        | <code>char</code>    | 16 bits     | <code>\u0000</code> (0)            | <code>\uffff</code> (65,535)      | <code>\u0000</code> |
| <b>Logical</b>        | <code>boolean</code> | 1 bit       | false                              | true                              | false               |

## 3.3 2. Two's Complement Formula & Range Bounds

Signed integers in Java use **Two's Complement** binary representation. For an  $n$ -bit signed integer:

$$\text{Range} = [-2^{n-1}, 2^{n-1} - 1]$$

- E.g., for `int` ( $n = 32$  bits):

$$\text{Min} = -2^{31} = -2,147,483,648$$

$$\text{Max} = 2^{31} - 1 = 2,147,483,647$$

### 3.3.1 Executable Code Example: Overflow Wrap-Around & Type Conversion

```
public class DatatypesDemo {
    public static void main(String[] args) {
        // 1. Integer Overflow Wrap-Around
        int maxInt = Integer.MAX_VALUE; // 2147483647
        System.out.println("Max Int Value: " + maxInt);
        System.out.println("Max Int + 1 (Overflow!): " + (maxInt + 1)); // Wraps to -2147483648

        // 2. Implicit Widening (int to double)
    }
}
```

```

    int count = 25;
    double price = count; // Automatic widening
    System.out.println("Widened Double Value: " + price);

    // 3. Explicit Narrowing (double to int casting)
    double pi = 3.9999;
    int roundedPi = (int) pi; // Truncates decimal part
    System.out.println("Casted Int (Truncated): " + roundedPi);

    // 4. Unicode Char Representation
    char tamilLetter = ' ';
    System.out.println("Unicode Char: " + tamilLetter + " | Int Code: " + (int) tamilLetter);
}

DatatypesDemo.main(new String[]{});

```

### 3.4 3. Comprehensive Summary & Key Takeaways

1. **8 Primitives:** byte, short, int, long, float, double, char, boolean stored directly on the Stack.
2. **Overflow Wrap-Around:** Arithmetic exceeding integer bounds wraps around to negative bounds silently.
3. **Boolean Safety:** Java boolean is not an integer, preventing accidental assignment-in-conditional bugs.

## 4 IITM Java Course — Week 2, Lecture 9: Control Flow Constructs in Java

**Instructor:** Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras)

**Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

### 4.1 Key Technical Terms Dictionary (Defined Upon First Appearance)

- **Control Flow:** The order in which individual statements, instructions, or function calls are executed or evaluated in a program.
- **Entry-Controlled Loop:** A loop structure (**while**, **for**) where the condition is evaluated **before** executing the loop body.
- **Exit-Controlled Loop:** A loop structure (**do-while**) where the condition is evaluated **after** executing the loop body, guaranteeing at least one execution.
- **Switch Fall-Through:** The behavioral mechanic in traditional **switch** statements where execution flows sequentially into subsequent cases unless explicitly terminated with **break**.
- **Enhanced For-Each Loop:** A simplified iteration syntax (**for (ElementType item : collection)**) introduced to traverse arrays and collections without manual index counters.

#### 4.1.1 Prof. Mukund's Words of Wisdom & Best Practices (Lecture 9)

**1. The Switch Fall-Through Trap:** \* *"In traditional Java **switch** statements, forgetting a **break**; statement does not cause a syntax error! Execution will fall through into the next case automatically. Always include explicit **break**; or use modern arrow syntax to prevent silent logic errors."*

**2. When to Use do-while Loops:** \* *"Use a **do-while** loop specifically when you need to perform an action at least once before asking the user for validation (e.g. prompting for a password or menu choice). Using a standard **while** loop forces duplicating the prompt code outside the loop."*

**3. Off-by-One Loop Bounds:** \* *"In 0-indexed arrays, iterating up to **<= array.length** causes an **ArrayIndexOutOfBoundsException**. Always use strictly **< array.length** or enhanced **for-each** loops."*

## 4.2 1. Conditional Branching (if, else if, switch)

Java evaluates logical expressions strictly returning boolean values (true or false).

```
graph TD;
    A["Check Condition (x > 0)"] -->|True| B["Execute Positive Block"];
    A -->|False| C["Check Condition (x == 0)"];
    C -->|True| D["Execute Zero Block"];
    C -->|False| E["Execute Negative Block"];
```

### 4.2.1 Executable Code Example: Conditionals, Switch, and Loop Constructs

```
public class ControlFlowDemo {
    public static void main(String[] args) {
        // 1. Switch Statement with Case Execution
        int dayNumber = 3;
        String dayName;
        switch (dayNumber) {
            case 1: dayName = "Monday"; break;
            case 2: dayName = "Tuesday"; break;
            case 3: dayName = "Wednesday"; break;
            default: dayName = "Invalid Day"; break;
        }
        System.out.println("Day #" + dayNumber + " is: " + dayName);

        // 2. Entry-Controlled Loop (while)
        int count = 1;
        System.out.print("While Loop: ");
        while (count <= 3) {
            System.out.print(count + " ");
            count++;
        }
        System.out.println();

        // 3. Exit-Controlled Loop (do-while guarantees at least 1 run)
        int val = 100;
        System.out.print("Do-While Loop Execution: ");
        do {
            System.out.print("Executed once with val=" + val);
        } while (val < 0); // Condition is false, but body executed once!
        System.out.println();

        // 4. Enhanced For-Each Loop over Array
        int[] primeNums = {2, 3, 5, 7, 11};
        int sum = 0;
        for (int p : primeNums) {
            sum += p;
        }
        System.out.println("Sum of Primes (Enhanced Loop): " + sum);
    }
}

ControlFlowDemo.main(new String[]{});
```

## 4.3 2. Comprehensive Summary & Key Takeaways

1. **Branching:** if-else and switch direct program flow based on boolean expressions.

2. **Looping:** `while` and `for` are entry-controlled; `do-while` is exit-controlled.
3. **Enhanced Loop:** `for (T item : collection)` avoids manual index management bugs.

## 5 IITM Java Course — Week 2, Lecture 10: Defining Classes and Objects in Java

**Instructor:** Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras)

**Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)

---

### 5.1 Key Technical Terms Dictionary (Defined Upon First Appearance)

- **this Keyword:** A reference pointer inside an instance method or constructor that refers directly to the invoking object itself.
- **Parameter Shadowing:** A variable scoping situation where a local parameter or variable name matches an instance field name, masking the instance field within that scope.
- **Constructor Overloading:** Defining multiple constructors within the same class template, differentiated by parameter count or types.
- **Method Signature:** The combination of a method's name and its ordered parameter list (type sequence) used by the compiler to distinguish methods.
- **Default Constructor:** A zero-argument constructor provided automatically by the Java compiler if no explicit constructors are defined in a class.

#### 5.1.1 Prof. Mukund's Words of Wisdom & Best Practices (Lecture 10)

**1. The Parameter Shadowing Pitfall:** \* *"In constructors, if you name parameters the same as your instance variables (`public Account(double balance)`), writing `balance = balance;` does NOTHING! It assigns the parameter to itself. Always use `this.balance = balance;` to explicitly target the instance field."*

**2. Why Constructors Have No Return Type:** \* *"Constructors look like methods, but they are initialization routines. They implicitly return a reference pointer to the newly allocated memory cell on the Heap. Writing a return type (like `void Account()`) turns it into a regular method, breaking object instantiation!"*

**3. The Automatic Default Constructor Rule:** \* *"Java supplies a zero-argument default constructor ONLY if you write no constructors. The moment you define even one custom parameterized constructor, Java withdraws the default constructor. Always define an explicit default constructor if required by frameworks."*

### 5.2 1. Class Structure: Instance Variables & Methods

A class encapsulates state (fields) and behavior (methods) into a reusable blueprint.

#### 5.2.1 Executable Code Example: Bank Account Class with Overloaded Constructors & Methods

```
class BankAccount {
    private String accountNumber;
    private double balance;

    // Constructor 1: Default Zero-Argument
    public BankAccount() {
        this.accountNumber = "UNKNOWN";
        this.balance = 0.0;
    }
}
```

```

// Constructor 2: Parameterized (Overloaded)
public BankAccount(String accountNumber, double initialBalance) {
    this.accountNumber = accountNumber; // Resolves Parameter Shadowing using 'this'
    this.balance = initialBalance;
}

// Deposit Method 1: Standard deposit
public void deposit(double amount) {
    if (amount > 0) {
        this.balance += amount;
    }
}

// Deposit Method 2: Overloaded deposit with bonus code
public void deposit(double amount, double bonus) {
    if (amount > 0) {
        this.balance += (amount + bonus);
    }
}

public double getBalance() {
    return this.balance;
}
}

public class ClassDefinitionDemo {
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount();
        BankAccount acc2 = new BankAccount("ACC-9988", 500.0);

        acc2.deposit(200.0); // Invokes Deposit Method 1
        acc2.deposit(100.0, 15.0); // Invokes Overloaded Deposit Method 2

        System.out.println("Account 1 Balance: $" + acc1.getBalance());
        System.out.println("Account 2 Balance: $" + acc2.getBalance());
    }
}

ClassDefinitionDemo.main(new String[]{});

```

### 5.3 2. Comprehensive Summary & Key Takeaways

1. **this Reference:** Disambiguates instance variables from local parameter shadowing.
2. **Overloading:** Multiple methods or constructors can share a name if parameter signatures differ.
3. **Constructors:** Executed automatically during `new` dynamic heap allocation.

## 6 IITM Java Course — Week 2, Lecture 11: Basic Input and Output in Java

**Instructor:** Prof. Madhavan Mukund (Chennai Mathematical Institute / IIT Madras)

**Course:** Programming Concepts Using Java (B.Sc in Programming and Data Science)



## 6.1 Key Technical Terms Dictionary (Defined Upon First Appearance)

- **java.util.Scanner:** A text-parsing utility class in Java used to parse primitive types and strings using regular expressions from input streams.
- **System.in:** The standard input stream connected to keyboard input by the operating system.
- **Input Buffer:** A temporary memory buffer holding raw character data typed by the user before it is consumed by input reader methods.
- **Trailing Newline Trap:** A common bug where `nextInt()` leaves the newline character (`\n`) in the input buffer, causing a subsequent `nextLine()` to consume the empty newline immediately.
- **System.out.printf():** A formatted output method supporting format specifiers (`%d`, `%f`, `%s`) for precision control.

### 6.1.1 Prof. Mukund's Words of Wisdom & Best Practices (Lecture 11)

**1. The Trailing Newline Trap in Scanner:** \* “When you call `sc.nextInt()`, Scanner reads the integer number but leaves the Enter key newline character (`\n`) sitting inside the input buffer! Calling `sc.nextLine()` immediately afterwards will read that leftover newline and return an empty string. Always consume the leftover newline (`sc.nextLine()`) before reading string input.”

**2. Resource Closing Best Practice:** \* “Scanner objects opened over `System.in` hold underlying system stream handles. Always close your scanners (`sc.close()`) or use try-with-resources to prevent system resource leaks.”

## 6.2 1. Reading User Input with Scanner & Formatted Output

### 6.2.1 Format Specifiers Matrix for System.out.printf():

- `%d`: Signed decimal integer.
- `%f`: Floating-point number (e.g. `%.2f` for 2 decimal places).
- `%s`: Character string.
- `%c`: Character.
- `%n`: Platform-independent line separator.

### 6.2.2 Executable Code Example: Scanner Parsing & Formatted Output

```
import java.util.Scanner;

public class InputOutputDemo {
    public static void main(String[] args) {
        // Simulating Input Stream via Scanner string constructor
        String simulatedInput = "42\nDevendra\n98.75\n";
        Scanner sc = new Scanner(simulatedInput);

        // 1. Parse Integer
        int age = sc.nextInt();
        sc.nextLine(); // CONSUME TRAILING NEWLINE BUFFER (Prof. Mukund's Tip!)

        // 2. Parse Full Line String
        String name = sc.nextLine();

        // 3. Parse Double
        double score = sc.nextDouble();

        // 4. Formatted Output Print
        System.out.printf("Student Record: %s | Age: %d | Score: %.2f%%\n", name, age, score);

        sc.close();
    }
}
```

```
}  
  
InputOutputDemo.main(new String[]{});
```

## 6.3 2. Comprehensive Summary & Key Takeaways

1. **Scanner Parsing:** Parses input streams into `int`, `double`, and `String` tokens.
2. **Trailing Newline Trap:** Clear input buffer with `sc.nextLine()` after `nextInt()` calls.
3. **Formatted Printing:** Use `System.out.printf()` for decimal precision control.